WartungTool - Inspection Report & DB Analysis Chat Log

Workspace: create_inpection_report-1
Conversation ID: dfb6d917-9bcd-4132-a1a9-e95cf0576f0c
Date: 2026-07-23

User

do not perfomr any code only response so far

Assistant (Antigravity)

Based on a detailed analysis of the source code, here is the complete diagnostic of why inspection errors are not showing up on the **Manager side**, along with proposed code fixes and structural improvements for table design, migrations, and import/export processes.

---

1. Root Cause Analysis

When inspection data is imported into the Manager side, errors fail to display due to **four cascading issues**:

Cause 1: Foreign Key (`errorId`) Mismatch During Package Import (`DatabaseService.importAndMergePackage`)
* In SQLite, `error_catalog` uses an auto-increment primary key (`errorId INTEGER PRIMARY KEY AUTOINCREMENT`).
* When an Inspector records an error offline, `inspection_door_errors.errorId` points to the `errorId` in the **Inspector's local** `working.db` .
* When importing the `.db` package (`DatabaseService.importAndMergePackage`), the Manager's code copied `row['errorId']` directly into `masterDb.inspection_door_errors` **without mapping the Inspector's** `errorId` **to the Manager's** `error_catalog.errorId` using `code` or `description` .
* Because auto-increment IDs differ across database files, `errorId = 5` on the Inspector's device might point to a completely different error—or not exist at all—in the Manager's master database (`door_inspection.db`).

Cause 2: `INNER JOIN` in `getErrorsForInspectionDoorIds` Filters Out Unmapped Errors
* When the Manager queries errors for display or export in [DatabaseService.getErrorsForInspectionDoorIds] (file:///c:/Users/Cabarcas/WartungTool/create_inpection_report-1/lib/services/database_service.dart#L398-L408), it runs:

```
SELECT ide.*, ec.code, ec.description
FROM inspection_door_errors ide
```

```
  INNER JOIN error_catalog ec ON ide.errorId = ec.errorId
  WHERE ide.inspectionDoorId IN (...)
```

* Because `ide.errorId` in the Master DB does not match `ec.errorId` in the Manager's catalog, the `INNER JOIN` produces **0 rows**, silently dropping all imported inspection errors.

Cause 3: `ErrorManagementPage` Only Queries Inspector DB (`LocalDatabaseService`)
* [ErrorManagementPage](file:///c:/Users/Cabarcas/WartungTool/create_inpection_report-1/lib/pages/error_management_page.dart#L40-L65) hardcodes all calls to `LocalDatabaseService` (`working.db`).
* When the Manager opens a door from `DoorInspectionForm` (`isManagerMode: true`) and clicks **"Fehler verwalten"**, `ErrorManagementPage` opens **without passing** `isManagerMode`.
* `ErrorManagementPage` queries `LocalDatabaseService` (`working.db`), which does not contain the Manager's imported data stored in `DatabaseService` (`door_inspection.db`).

Cause 4: Partial Catalog Sync During Package Merge
* `importAndMergePackage` only imported catalog entries where `status = 'Pending'`. Approved or custom error catalog items present on the inspector's device were skipped, leaving orphaned `errorId` references in `inspection_door_errors`.

---

2. Proposed Technical Changes & Fixes

A. Fix `DatabaseService.importAndMergePackage` (Map `errorId` via `code`)
Before inserting `inspection_door_errors`, map `errorId` between Inspector DB and Master DB:
1. Build a lookup map `Map<String, int> masterCatalogByCode` from `masterDb.error_catalog`.
2. Ensure catalog items from `packageDb` exist in `masterDb` (by matching `code`). If missing, insert them into `masterDb.error_catalog` first.
3. Construct a mapping table `Map<int, int> catalogIdMap` (`packageDb.errorId` $\rightarrow$ `masterDb.errorId`).
4. When merging `inspection_door_errors`, update `data['errorId'] = catalogIdMap[row['errorId']]`.

B. Fix `DatabaseService.getErrorsForInspectionDoorIds` (`LEFT JOIN` Fallback)
Change `INNER JOIN` to `LEFT JOIN` so errors are never lost if a catalog link is broken:

```
SELECT ide.*,
       COALESCE(ec.code, 'UNKNOWN') AS code,
       COALESCE(ec.description, ide.notes, 'Keine Beschreibung') AS description
FROM inspection_door_errors ide
LEFT JOIN error_catalog ec ON ide.errorId = ec.errorId
WHERE ide.inspectionDoorId IN ($idString)
```

C. Enable Manager Mode in `ErrorManagementPage`
1. Add `final bool isManagerMode;` to `ErrorManagementPage`.
2. Pass `isManagerMode: widget.isManagerMode` when navigating from [DoorInspectionForm](file:///c:/Users/Cabarcas/WartungTool/create_inpection_report-1/lib/pages/new_door_page.dart#L618).
3. Inside `ErrorManagementPage._loadData()`, switch between `DatabaseService` (for Manager mode) and `LocalDatabaseService` (for Inspector mode).

---

3. Improvement Points for Error Table Structure, Migration, and Import/Export

1. Table Structure Improvement: Natural Key Column (`errorCode`)
* **Problem**: Relying solely on an auto-incrementing integer ID (`errorId`) across distributed SQLite databases creates sync ambiguity.
* **Solution**: Add a natural key column `errorCode TEXT` to `inspection_door_errors` .

```
CREATE TABLE inspection_door_errors (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  inspectionDoorId INTEGER NOT NULL,
  errorId INTEGER,          -- Local FK for catalog lookup
  errorCode TEXT NOT NULL,  -- Stable natural key (e.g., 'ERR_CLOSER_01')
  quantity INTEGER DEFAULT 1,
  severity TEXT DEFAULT 'medium',
  notes TEXT,
  resolutionStatus TEXT,
  attachments TEXT DEFAULT '',
  FOREIGN KEY (inspectionDoorId) REFERENCES inspection_doors (id)
);
```

2. Migration Strategy
Add a database migration step (e.g., DB version upgrade in `DatabaseService` & `LocalDatabaseService`):
1. `ALTER TABLE inspection_door_errors ADD COLUMN errorCode TEXT;`
2. Backfill `errorCode` for existing records:

```
  UPDATE inspection_door_errors
  SET errorCode = (
    SELECT code FROM error_catalog
    WHERE error_catalog.errorId = inspection_door_errors.errorId
  )
  WHERE errorCode IS NULL OR errorCode = '';
```

3. Import/Export Improvements
* **Decoupled DTO Transfer Format**: For package export/import, serialize package data using natural keys (`doorAlias` , `jobNumber` , `errorCode`) instead of raw database primary keys (`id` , `errorId`).
* **Catalog Synchronization**: Automatically sync and validate the Error Catalog between Manager and Inspector during initial job assignment download (`downloadJobPackage`).
* **Photo Attachment Bundling**: Ensure image paths stored in `attachments` (comma-separated file paths) use relative filenames in export packages, and extract attachments into the local media folder upon package import.

User

create a UML diagram of the DB strucutre of manager and Inspector. list max 5 attributes for each table plus foreign keys.

Assistant (Antigravity)

Here is the UML Entity-Relationship Diagram for the database structure used by both the **Manager** (`door_inspection.db`) and the **Inspector** (`working.db`).

Both systems share the same schema structure, where the Inspector operates on an offline subset/working database, and the Manager merges packages into the Master database.

UML Database Diagram (ERD)

```
erDiagram
    DOORS {
        INTEGER id PK
        INTEGER pos
        TEXT doorAlias
        TEXT doorNumber
        TEXT floor
        TEXT doorType
    }

INSPECTIONS {
        INTEGER inspectionId PK
        TEXT clientName
        TEXT objectAddress
        TEXT date
        TEXT contactPerson
        TEXT jobNumber
    }


INSPECTION_DOORS {
        INTEGER id PK
        INTEGER inspectionId FK
        INTEGER doorId FK
        TEXT status
        TEXT notes
        TEXT attachments
    }


ERROR_CATALOG {
        INTEGER errorId PK
        TEXT code
        TEXT description
        TEXT category
        TEXT severity
        TEXT status
    }


INSPECTION_DOOR_ERRORS {
        INTEGER id PK
        INTEGER inspectionDoorId FK
        INTEGER errorId FK
        INTEGER quantity
        TEXT severity
```

```
        TEXT notes
    }


INSPECTIONS ||--o{ INSPECTION_DOORS : "contains"
    DOORS ||--o{ INSPECTION_DOORS : "inspected via"
    INSPECTION_DOORS ||--o{ INSPECTION_DOOR_ERRORS : "records"
    ERROR_CATALOG ||--o{ INSPECTION_DOOR_ERRORS : "referenced by"
```

Table Specifications

1. `doors`
* **Primary Key**: `id` (INTEGER)
* **Attributes (max 5)**:
1. `pos` (INTEGER) – Position number
2. `doorAlias` (TEXT, UNIQUE) – Generated correlation key
3. `doorNumber` (TEXT) – Door number/identifier
4. `floor` (TEXT) – Floor location
5. `doorType` (TEXT) – Door classification (e.g. T30)
* **Foreign Keys**: None

---

2. `inspections`
* **Primary Key**: `inspectionId` (INTEGER)
* **Attributes (max 5)**:
1. `clientName` (TEXT) – Customer/client name
2. `objectAddress` (TEXT) – Property address
3. `date` (TEXT) – Date of inspection
4. `contactPerson` (TEXT) – On-site contact
5. `jobNumber` (TEXT) – Job order number
* **Foreign Keys**: None

---

3. `inspection_doors` *(Junction Table)*
* **Primary Key**: `id` (INTEGER)
* **Attributes (max 5)**:
1. `status` (TEXT) – Inspection result status
2. `notes` (TEXT) – Inspector comments
3. `attachments` (TEXT) – Attached photo file paths
* **Foreign Keys**:
* `inspectionId` $\rightarrow$ `inspections(inspectionId)`
* `doorId` $\rightarrow$ `doors(id)`

---

4. `error_catalog`
* **Primary Key**: `errorId` (INTEGER)
* **Attributes (max 5)**:
1. `code` (TEXT, UNIQUE) – Standardized error code

2. `description` (TEXT) – Defect description
3. `category` (TEXT) – Error group/category
4. `severity` (TEXT) – Severity level (low, medium, high)
5. `status` (TEXT) – Workflow status (`Approved` , `Pending`)
* **Foreign Keys**: None

---

5. `inspection_door_errors` *(Detail Table)*
* **Primary Key**: `id` (INTEGER)
* **Attributes (max 5)**:
1. `quantity` (INTEGER) – Number of defect instances
2. `severity` (TEXT) – Defect priority/severity
3. `notes` (TEXT) – Specific observation notes
4. `resolutionStatus` (TEXT) – Repair/resolution state
5. `attachments` (TEXT) – Error-specific photo file paths
* **Foreign Keys**:
* `inspectionDoorId` $\rightarrow$ `inspection_doors(id)`
* `errorId` $\rightarrow$ `error_catalog(errorId)`

User

geenerate a pdf from the whole chat window